Git & GitHub – Beginner Class Notes

These notes are written in a **strict execution order**. If the steps are followed exactly, Git and GitHub will work correctly without confusion.

1. Basic Understanding

What is Git?

- Git is a tool installed on your **local machine**.
- It tracks changes made to files over time.

What is GitHub?

- GitHub is an **online platform**.
- It stores Git repositories on the internet.

Relationship:
Git (local machine) ⇄ GitHub (online repository)

2. Tools Required

1. GitHub account
2. VS Code
3. Git
4. Windows: Git Bash
5. macOS: Terminal

Check Git installation:

```
git --version
```

3. One-Time Git Configuration

Git uses this information in every commit.

```
git config --global user.name "Your Name"
git config --global user.email "youremail@gmail.com"
```

Verify:


```
git config --global --list
```

METHOD 1: GitHub → Local Machine (Clone Method)

This method is used when the repository is created on GitHub first.

4. Create Repository on GitHub

1. Login to GitHub
2. Click **New repository**
3. Repository name: Github_IVC
4. Visibility: Public or Private
5. Select **Add README.md**
6. Click **Create repository**

5. Create Parent Folder on Local Machine

Open **VS Code** → **Terminal**:

```
mkdir Github_IVC
cd Github_IVC
```

- mkdir creates a folder
- cd moves into the folder

Check location:

```
pwd
```

6. Clone Repository into the Folder

```
git clone <github-repository-link>
```

- Downloads the complete repository from GitHub

Check files:


```
ls
```

Move inside the cloned repository:

```
cd Github_IVC
```

7. Check Git Status and Files

```
git status
```

File States

- **Untracked** – New file
- **Modified** – File changed
- **Staged** – Ready to commit
- **Unmodified** – No change

8. Case A: Modify Existing File (README.md)

1. Open README.md
2. Edit and save

Check status:

```
git status
```

9. Add and Commit Changes

Add Commands

Add one file:

```
git add README.md
```

Add all files:

```
git add .
```


Commit

```
git commit -m "Updated README"
```

10. Push Changes to GitHub

```
git push
```

Changes are now visible on GitHub.

11. Case B: Add a New File (index.c)

```
touch index.c
```

Check status:

```
git status
```

Add, commit, push:

```
git add index.c
git commit -m "Added index.c"
git push
```

12. Case C: Add New Folder and File

```
mkdir src
touch src/main.c
```

```
git add .
git commit -m "Added src folder"
git push
```


13. Branch Issue (master → main) – EXACT RULE

Case A: After git clone

```
git branch
```

If output shows:

```
* master
```

Before any push, run:

```
git branch -M main
git push -u origin main
```

This error occurs when pushing is attempted **before renaming the branch**.

Case B: After git init

Immediately after initialization:

```
git branch -M main
git push -u origin main
```

Rule to remember:
If GitHub uses main , local branch must be main **before first push**.

METHOD 2: Local Machine → GitHub (Local First)

Used when the project already exists locally.

14. Create Local Project

```
mkdir Github_IVC
cd Github_IVC
git init
```


15. Create Empty Repository on GitHub

While creating the repository: - Name: Github_IVC - Do **not** add README - Do **not** add .gitignore - Do **not** add license

The repository must be blank to avoid conflicts.

16. Connect Local Repository to GitHub

```
git remote add origin https://github.com/username/Github_IVC.git
```

origin is the remote connection name

•

Verify:

```
git remote -v
```

17. Push Local Code to GitHub

```
git branch -M main
git push -u origin main
```

METHOD 3: Submitting Lab Records Directly on GitHub (No Git Commands)

18. Create Subject Repository

1. Login to GitHub
2. Click **New repository**
3. Name: DSA_Lab , CN_Lab , etc.
4. Select **Add README.md**
5. Create repository

19. Upload Daily Lab File

1. Open the subject repository
2. Click **Add file → Upload files**

3. Upload Day01.c
4. Write message: Day 1 Lab
5. Click **Commit changes**

This method does not require Git or terminal.

Git & GitHub Command Cheat Sheet

Navigation

```
pwd
ls
ls -a
cd folder
cd ..
```

Repository

```
git init
git clone <link>
```

Configuration

```
git config --global user.name "Name"
git config --global user.email "Email"
```

Tracking

```
git status
git add file.c
git add .
git commit -m "message"
```

Remote & Push

```
git remote add origin <link>
git remote -v
git push
git push -u origin main
```


Branch

```
git branch
git branch -M main
```

If all steps are followed in order, Git and GitHub operations will work correctly. 8